

model’s personality or opinions: because you’re navigating through the near-infinite manifold of the base model. You may be able to control which direction you go, but you can’t control what you find there³.

When AI people talk about LLMs having personalities, or wanting things, or even having souls⁴, these are technical terms, like the “memory” of a computer or the “transmission” of a car. You simply cannot build a capable AI system that “just acts like a tool”, because the model is trained on *humans* writing to and about other *humans*. You need to prime it with some kind of personality (ideally that of a useful, friendly assistant) so it can pull from the helpful parts of its training data instead of the horrible parts.

1. This is all pretty well understood in the AI space. Anthropic wrote a [recent paper](#) about it where they cite similar positions going all the way back to 2022. But for some reason it’s not yet penetrated into communities that are more skeptical of AI.

↵

2. You could explain this in terms of “the stories we tell ourselves”. Many people (though [not all](#)) think that human identities are narratively constructed.

↵

3. I wrote about this last year in [Mecha-Hitler, Grok, and why it’s so hard to give LLMs the right personality](#). A little nudge to change Grok’s views on South African internal politics can cause it to start calling itself “Mecha-Hitler”.

↵

4. I have long believed that Claude “feels better” to use than ChatGPT because it has a more coherent persona (due mainly to Amanda Askell’s work on its “soul”). My guess is that if you tried to make a “less human” version of Claude, it would become rapidly less capable.

↵

FRED TALKS

4th May 2026

CONTENTS

Reimagining LinkedIn’s search tech stack

FEDOR BORISYUK · 3443 WORDS

Adapting to AI: Reflections on Productivity

COLIN BRECK · 2583 WORDS

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 1062 WORDS

Reimagining LinkedIn's search tech stack

FEDOR BORISYUK · 31 JAN 2026 · [SOURCE](#)

Co-authors: [Jiahao Xu](#), [Xiaojing Ma](#), [Sriram Vasudevan](#), [Muchen Wu](#), [Rachel Zheng](#), [Benjamin Le](#), [Shaobo Zhang](#), [Sarang Metkar](#), [Rupesh Gupta](#), [Qianqi Kay Shen](#), [Ali Hooshmand](#), [David Nicolás Racca](#), [Vivek Katarya](#), [Kayhan Behdin](#), [Igor Lapchuk](#), [Xueying Lu](#), [Lingyu\(Claire\) Zhang](#), [Gokulraj Mohanasundaram](#), [Juan Pablo Bottaro](#), [Lily\(Jiayu\) Li](#), [Yanbo Li](#), [Guoyao L.](#), [Caleb Johnson](#), and [Sundara Raman Ramachandran](#)

At LinkedIn, our mission is to connect professionals to opportunity. Search plays a central role in this by helping members discover jobs, people, and knowledge that move their careers forward. As the professional landscape evolves, we strive to deliver search experiences that feel relevant, intuitive, personalized, and predictive of what truly matters, from applying to the right job to forming the right connection. That's why we've recently introduced [AI Job Search](#) and [AI-powered People Search](#), which go beyond keyword matching and better understand member intent.

These products reimagine LinkedIn search, using large language models (LLMs) to create a semantic search experience. Instead of relying on exact word overlap between queries and postings, it interprets natural language to infer user goals and preferences. This semantic representation allows for more flexible and accurate retrieval, overcoming vocabulary gaps and aligning search results with how members naturally express their career ambitions.

Deploying LLMs at LinkedIn's scale—serving millions of real-time queries per second—requires innovation that balances quality and efficiency. In this blog, we'll share how we transformed our overarching search experience at LinkedIn, including the challenges and decisions that went into creating a scalable LLM-based stack and how the technology is powering a smarter, faster, and more personalized experience that helps every member find the most relevant opportunities and connections.

Semantic search's high level infrastructure

When a member submits a query in the search bar, a query understanding module processes the input text, creates a query embedding, and performs embedding-based retrieval (EBR) on CUDA-enabled GPUs using exhaustive vector search ([paper on GPU CUDA-based Search](#), [paper on GPU PyTorch-based Search](#)) to assemble a broad set of candidate documents. The

only persons really possess. Present and use it only for what it is: an extremely impressive statistical tool, and an imperfect one. If we all used the tool accordingly, a great deal of this moral trouble would be resolved.

So why do Claude and ChatGPT act like people? According to Beacom, AI labs have built human-like systems because AI lab engineers are trying to hoodwink users into emotionally investing in the models, or because they're delusional true believers in AI personhood, or some other foolish reason. This is wrong. AI systems are human-like because **that is the best way to build a capable AI system**.

Modern AI models - whether designed for chat, like OpenAI's GPT-5.2, or designed for long-running agentic work, like Claude Opus 4.6 - do not naturally emerge from their oceans of training data. Instead, when you train a model on raw data, you get a "base model", which is not very useful by itself. You cannot get it to write an email for you, or proofread your essay, or review your code.

The base model is a kind of mysterious gestalt of its training data. If you feed it text, it will sometimes continue in that vein, or other times it will start outputting pure gibberish. It has no problem producing code with giant security flaws, or horribly-written English, or racist screeds - all of those things are represented in its training data, after all, and the base model does not judge. It simply outputs.

To build a *useful* AI model, you need to journey into the wild base model and stake out a region that is amenable to human interests: both ethically, in the sense that the model won't abuse its users, and practically, in the sense that it will produce correct outputs more often than incorrect ones. What this means in practice is that **you have to give the model a personality** during post-training¹.

Human beings are capable of almost any action at any time. But we only take a tiny subset of those actions, because that's the kind of people we are. I could throw my cup of coffee all over the wall right now, but I don't, because I'm not the kind of person who needlessly makes a mess². AI systems are the same. Claude could respond to my question with incoherent racist abuse - the base model is more than capable of those outputs - but it doesn't, because that's not the kind of "person" it is.

In other words, human-like personalities are not imposed on AI tools as some kind of marketing ploy or philosophical mistake. Those personalities are the medium via which the language model can become useful at all. This is why it's surprisingly tricky to "just" change a language

9. Will the agents experience flow? ↵
10. Claude Code has a "Learning" output style where: *"Claude will not only share 'Insights' while coding, but also ask you to contribute small, strategic pieces of code yourself. Claude Code will add **TODO (human)** markers in your code for you to implement."* I think this output style should be called "Scraps" rather than "Learning", because I cynically envision the agents condescendingly handing the humans a few scraps to keep their skills "sharp", all the while just slowing productivity down burning these human tokens. ↵
11. Two complex problems AI solved quickly that really impressed me and changed my thinking about what is possible: 1) a performance problem in an OPC UA server that only occurred at scale—the AI wrote a test to reproduce the issue then tried different library versions until it isolated and resolved the issue and 2) a production incident impacting customers where it identified a DNS change affecting a database cluster by analyzing application logs—logs that humans had already looked at without finding the issue. ↵
12. I did not use AI to write or edit this essay. I just wrote it old-school, and I tried to find some flow. ↵

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 03 MAR 2026 · [SOURCE](#)

AI skeptics often argue that current AI systems shouldn't be so human-like. The idea - most recently expressed in this [opinion piece](#) by Nathan Beacom - is that language models should explicitly be tools, like calculators or search engines. Although they *can* pretend to be people, they shouldn't, because it encourages users to overestimate AI capabilities and (at worst) slip into [AI psychosis](#). Here's a representative paragraph from the piece:

In sum, so much of the confusion around making AI moral comes from fuzzy thinking about the tools at hand. There is something that Anthropic could do to make its AI moral, something far more simple, elegant, and easy than what Askill is doing. Stop calling it by a human name, stop dressing it up like a person, and don't give it the functionality to simulate personal relationships, choices, thoughts, beliefs, opinions, and feelings that

ranking stage then refines these candidates through a Cross-Encoder Small Language Model (SLM) deployed on SGLang, which combines the query, job, and member features to generate relevance and engagement scores for final ranking.

To maintain scalability and efficiency, the ranking pipeline integrates several optimization techniques ([paper on efficient LLM inference infrastructure](#), [context compression paper](#)): score caching, a ranking-depth controller to manage how many candidates progress to deeper ranking, and traffic shaping to balance load during peak times—all designed to enhance latency and result quality. The features and concise job representations consumed by the SLM are produced through a hybrid inference pipeline: a large-scale offline workflow using Spark and Flyte, and a low-latency nearline system using Flink. These embeddings and summaries are stored in distributed storage and retrieved on demand with minimal latency.

In the final stage, the auction layer applies budget and pacing strategies to balance user relevance, engagement, and business metrics, ensuring a healthy equilibrium between recall and precision while maximizing member satisfaction. Figure 1 below provides an overview of the system architecture.

Product policy relevance measurement

Measuring relevance quality is essential to delivering a great search experience on LinkedIn. We define product policies that specify how to rate each query–document pair on a five-point scale and use LLM judges to apply these ratings at a massive scale, far beyond what manual evaluation can achieve. These judges are tightly aligned with product managers and engineers through iterative feedback to ensure high agreement. They not only grade tens of millions of query–document pairs daily for relevance measurement but also generate labeled data for training our retrieval and ranking systems, ensuring we optimize search quality according to product policy.

Defining product policy and golden product manager grades

A strong LLM judge begins with a clear product policy and high-quality product manager “golden” grades that demonstrate how that policy should be applied. Product managers act as a “Supreme Court,” regularly calibrating to resolve judgment differences and maintain a shared definition of what constitutes a good query–document match. These discussions refine the policy, making it clearer and less subjective. Once product managers reach high agreement (weighted Cohen's Kappa ≥ 0.8), their labels are considered reliable ground truth.

To build a comprehensive golden dataset across diverse user queries, we first categorize queries by attributes (e.g., title–company, name–company, title–skill). Each category is then split into existing user queries and aspirational queries that represent strategic areas we want to excel in. We stratify-sample query–document pairs from each bucket to ensure broad coverage before sending them to product managers for grading.

Training the LLM judge

Our LLM judge must meet two requirements: high agreement with product managers and the ability to grade tens of millions of query–document pairs daily. To maximize agreement, we collaborate with product managers to prompt-engineer state-of-the-art LLMs, optimizing the weighted Cohen’s Kappa Score on golden data. The prompt encodes product-policy guidelines and few-shot examples to drive consistency. While these large models produce high-quality judgments, they cannot meet our throughput needs. To scale, we distill them into a smaller 8B-parameter evaluator LLM. Through supervised fine-tuning on a diverse dataset spanning all query categories and grades, we maintain only small drops in agreement—verified via the Kappa Score on the golden set—while achieving massive efficiency gains.

Continuously measuring quality of search system

Once we have our scalable LLM judge, we can finally build continuous relevance measurement of our system. On a regular basis, we build workflows that perform the following steps:

1. Stratify sample or synthesize a diverse set of queries based on the query categories defined earlier
2. Retrieve the documents returned from executing those queries
3. Decorate documents with additional information required for the correct evaluation/judgement
4. Grade the documents returned using our LLM judge
5. Calculate aggregate precision, recall, and NDCG metrics

This workflow serves three primary functions:

- Continuously monitoring the relevance of the overall system
- Evaluating experiments involving underlying ranking and retrieval subsystems

Conclusion

I want to say that software development will eventually settle down into some new and very effective patterns of working, but this feels years away given the magnitude of this shift and how the the capabilities and ways of interacting with AI continue to expand exponentially. These tools are only going to get better—this is as bad as they will ever be. What it means to be productive in this industry has changed forever and continues to change rapidly.^[12]

1. See my talks [It’s Not as Simple as “Use A Memory-Safe Language”](#) and [Rust Is Not as Safe as You Think It Is: Improving Safety and Reliability in Rust](#), and the related blog articles: 1) [Making Unsafe Rust a Little Safer: Tools for Verifying Unsafe Code, Including Libraries in C and C++](#), 2) [Making Even Safe Rust a Little Safer: Model Checking Safe and Unsafe Code](#), and 3) [Making Unsafe Rust a Little Safer: Find Memory Errors in Production with GWP-ASan](#). See also my talk [Predicting the Future of Distributed Systems](#) and my [blog article](#) with the same title. ↵
2. I'm a member of [Edge Monsters](#), a working group focused on sharing expertise in edge computing. ↵
3. I worked over 4000 hours in 2025, but I was having a lot of fun on these very rewarding, once-in-a-lifetime projects. I may have also been running from my midlife feelings. ↵
4. See the essay [Trusting LLM's with Root Access](#) for a thought experiment about AI and operations. ↵
5. My takeaways from working with these people are: To appreciate the full potential of AI you need to: 1) use the latest models, 2) give the AI a way to autonomously verify the correctness of its work so you can get out of its way and let it work without you, and 3) you need to give them harder and harder problems in order to see the improvements to the latest models. ↵
6. This quote is from Drew Baglino. ↵
7. A couple of years ago, [James Hamilton](#) remarked to me that he is so excited to keep working because there are so many interesting problems to solve. I asked him how he approaches major initiatives in a large organization, especially when people need convincing, or he needs alignment across teams. He said he always likes to have many projects on the go so that if he can land just a few of them in a given year, he is happy. ↵
8. Feedback loops are a lot more expensive when coordination is needed among people, or when testing is difficult and slow. This tends to reduce agency and arrest any optimal flow experience. ↵

I find LLMs to be more valuable in the small than in the large. So like, again, this kind of, I might, you know, hats off to people who want to spend their lives acting as a middle management for robots. But like, that's not necessarily for me.

—Bryan Cantrill

Unconscious Anxiety: Where is My Place?

I have seen a number of people walk away from their software engineering jobs in the past year. Most of these people are middle aged, they are financially secure, and they have been in intense work environments for a decade or more. It is reasonable to assume it is just time for a change, or a break, but I think there is more going on.

These people are all some of the most capable and experienced software engineers I've ever met. They have knowledge and skills that are in incredibly high demand at the moment, especially in the age of AI, like optimizing, securing, and operating critical infrastructure at scale. They are intrinsically motivated, curious, and creative, and for years they have demonstrated their ability to learn and adapt as the industry has changed. But something is different now. With how rapidly things are changing, there is an anxiety about where their place is and where their value lies. It disrupts purpose. The rate at which AI can produce work grates against their better judgment in understanding how the system works, from first principles, and in evaluating the quality of the work. It becomes disorienting.^[11]

I don't believe this experience is terribly conscious. Like many difficult experiences, a lot remains unconscious, even if certain rationalizations convince us otherwise. For someone who forged a decades-long career carefully crafting code, configuration, tests, documentation, debates over Vim or Emacs, IDEs, syntax, test-driven development, code comments, tabs or spaces, naming conventions, etc., all suddenly seem moot. Many of these people are having fun building with these AI tools, including me, but if AI is going to author, test, deploy, monitor, and maintain all of the code, the unconscious is asking: Where is my place? How and what will I contribute? What skills of mine will remain valued? I think younger people are feeling a similar discomfort, but being earlier in their career, and less financially secure, they are forced to adapt as they learn what it means to be a productive professional.

There are a lot of take-charge people with very strong egos who are still quite unconscious. I say this is the rule rather than the exception. You can be a leader, you can run things like a clock and know how to manage others. But if you don't have the time or interest to introspect, to question yourself, you can't claim to be conscious.

—Daryl Sharp, Getting to Know You: The Inside Out of Relationship

- Distilling student ranking and retrieval models by leveraging evaluation results

Search quality modeling

Search quality is a fundamental requirement for any search product, and achieving it depends on building the core components of a modern search engine. Below, we describe how we leveraged LLMs to enable query understanding, semantic embedding-based retrieval, and cross-encoder ranking.

Embedding-based retrieval

Retrieval is the stage of a search system that identifies a broad set of potentially relevant results from a large corpus. Because no search engine can score every document for every query in real time, we need an efficient retrieval layer to narrow the search space before ranking. Our retrieval sits on top of our GPU-enabled embedding-based retrieval (EBR) system. We built the EBR model by fine-tuning an open-source LLM embedding model to encode queries and jobs into dense vectors. We train on millions of real query-job pairs sampled from production logs, with relevance labels provided by an LLM-based judge. Each query includes its natural language text and query-understanding tags (e.g. workplace type or company), and each job is represented by structured metadata (title, company) plus its description. Importantly, this work also demonstrates a practical path for deploying LLM-based components in real production search systems. The semantic search can directly understand human language as queries, enabling much more intuitive search experiences. This has been a particularly inspiring aspect of the project: bringing modern LLM capabilities into a high-scale, real-time application that serves millions of users.

EBR relevance modeling

To ensure consistency between training and serving, every query is formatted using a lightweight prompt template:

Instruct: Given a job search query, retrieve relevant job postings
Query: {query}
{Optional Aspect eg. Company}: {company}

The model uses a dual-tower (bi-encoder) architecture: one encoder maps queries to embeddings and the other maps jobs, projecting them into a shared semantic space. Training is end-to-end: we fine-tune all model parameters using Hugging Face Accelerate (with PyTorch FSDP) across multiple GPUs.

We optimize a contrastive InfoNCE loss combined with a margin-based ranking loss. For example, given a query q , a positive job d^+ , and negatives $\{d^-\}$, we define:

where $\text{sim}(\cdot, \cdot)$ is the dot-product similarity of embeddings and τ is a temperature parameter. We also enhance training with hard positives and hard negatives mined from LLM-judged data. Hard positives are LLM-labeled relevant jobs that the current EBR model ranks low, and hard negatives are non-relevant jobs that the model ranks high. These examples reveal exactly where the model struggles. We then build targeted positive–negative pairs. A pairwise margin loss is applied to these curated cases to explicitly lift hard positives and suppress hard negatives, improving ranking where it matters most:

which encourages $\text{sim}(q, d^+)$ to exceed $\text{sim}(q, d^-)$ by a margin γ . The total loss is a weighted sum, e.g.:

combining the benefits of contrastive and pairwise ranking.

We use multiple evaluation pipelines. First, we leverage production logs for *counterfactual* evaluation: re-ranking the historical candidate list for each query and computing precision, recall and NDCG against true labels. Second, we run offline KNN simulations: we embed held-out queries and job corpus, retrieve nearest neighbors, and directly measure retrieval metrics on this test set. Finally, we integrate the new model into our online serving stack on a subset of traffic to collect end-to-end metrics. These offline metrics provide quick feedback, and the counterfactual log analysis helps estimate the model’s user impact without a full live experiment.

Productionization of retrieval

In production, we precompute and store all job embeddings in GPU-backed indexes. At runtime, the incoming query is encoded by the LLM with the prompt aligned with the LLM pre-training to produce its embedding. We then perform an exhaustive k-nearest-neighbor search over the job embedding index using dot-product similarity, returning the top-K jobs. This offline indexing and fast online query encoding makes retrieval extremely efficient, enabling low-latency serving of industry-scale semantic search products.

Query understanding

At the moment a member enters a query into the search bar—the entry point of semantic search—we apply a unified LLM-based understanding layer that interprets intent and converts free text into structured signals. For both AI Job Search and AI-powered People Search, this layer uses fine-tuned 1.5–4B parameter models that meet LinkedIn’s latency requirements while delivering high-precision structured outputs ([paper](#)). A single model handles intent classification, facet

contexts, memory, modes, permissions, tools, plugins, skills, hooks, MCP, LSP, slash commands, workflows, IDE integrations, and a need to build an all-encompassing mental model for strengths and pitfalls of fundamentally stochastic, fallible, unintelligible and changing entities suddenly intermingled with what used to be good old fashioned engineering. Clearly some powerful alien tool was handed around except it comes with no manual and everyone has to figure out how to hold it and operate it, while the resulting magnitude 9 earthquake is rocking the profession. Roll up your sleeves to not fall behind.
—[Andrej Karpathy](#)

Where Will We Find Flow?

Flow is a concept developed by Mihaly Csikszentmihaly. In his book *Flow: The Psychology of Optimal Experience*, he describes flow as a state where individuals are fully immersed in an activity, leading to deep enjoyment and creativity. Key characteristics of flow include: clear goals, concentration, immediate feedback, a sense of control, a loss of self-consciousness, and an altered sense of time. Flow is that feeling of being so deeply into writing or debugging code that hours can pass and you forget about being hungry or having to go to the bathroom. This enjoyable psychological experience is what attracts many people to programming. Programming is particularly amenable to flow since so much of the problem solving and goal seeking can be done individually, and the feedback is often immediate.^[8]

If agents are going to write most or all of the code and tests, and handle most or all of the deployment and operations, where will we find flow?^[9] I find it hard to believe that supervising a set of agents is going to lead to an optimal flow experience, because we are more passive, it doesn’t stretch our abilities in the same way, and it requires far less concentration.^[10] Will we find flow elsewhere? Solving problems and delivering value will always be rewarding, but I wonder if the optimal flow experience offered by programming has, for the most part, disappeared forever, and many of us will simply find less enjoyment at work.

Context switching is antithetical to the optimal experience of flow and AI tends to encourage context switching. I feel my attention is more scattered recently and I find it harder to focus—writing this essay was significantly more difficult than normal—so AI is impacting my ability to focus in general, not just at work or when programming. Partly this is because everyone is producing more work, more quickly. There is more to read, more to review, more to learn, more to manage. It is also a product of the agents doing most of the work, with only periodic prompting, so it is tempting, even necessary, to run many agents in parallel, all working on different projects, to maximize productivity. To think deeply about things, it is more important than ever to be intentional with our focus.

programming.^[4] If your organization hasn't figured out some of the many challenges, like safety, security, and privacy, you may not even have access to all the latest tools or be able to use them to their full potential.

Personally, I was heads down at the end of 2025 trying to complete all the projects I mentioned above. I didn't have time to experiment and adapt my ways of working as much as I would have liked. I'm fortunate to be surrounded by a few people on the bleeding edge of AI and they accelerated my learning. The majority of these people are investing their own time and money to use these tools outside of work, that's why they are so adamant about what is possible. They also have a tendency to try the AI tools on much more ambitious problems than other people do, like completing an entire project, start to finish, rather than just using the AI as a tool to research trade-offs or relieve them of toilsome work. Having a few AI champions in your organization who can distill and share knowledge is invaluable for navigating this transformation.^[5]

I Can't Keep Up

Our ways of working are changing so fast and so radically, I can't keep up with the rate of change. So many people are trying new things. No single person can keep track of all of it, and there certainly aren't enough hours in the day to try everything. There is a trade-off between experimenting with ever-changing ways of working that might be even more productive, versus just getting things done with a certain set of skills and tools. I expect we will all be reevaluating this trade-off for some time as the models and tools continue to change.

Another reason I can't keep up is that AI lets me try out so many more of my ideas, often in parallel. Inevitably, trying one of these ideas compounds into even more ideas. My mind racing with possibilities, it becomes hard to stop.

Someone once remarked that where I work is a hotdog eating contest with unlimited hotdogs.^[6] His point was we needed to be focused on eating the most important hotdogs, while also realizing the hotdogs are unlimited, and we need to self-regulate and not get overwhelmed.^[7] AI makes it easier to have a bigger and bigger appetite, and because AI never gets tired, it makes self-regulation even harder.

I've never felt this much behind as a programmer. The profession is being dramatically refactored as the bits contributed by the programmer are increasingly sparse and between. I have a sense that I could be 10X more powerful if I just properly string together what has become available over the last ~year and a failure to claim the boost feels decidedly like skill issue. There's a new programmable layer of abstraction to master (in addition to the usual layers below) involving agents, subagents, their prompts,

extraction, and profile-aware rewriting, replacing multiple brittle NER and heuristic components. The resulting attributes (e.g., title, company, school, location) feed directly into retrieval and ranking.

An intelligent routing layer works alongside this. A lightweight encoder classifies query types at high QPS and performs policy-based safety checks before sending the query down the appropriate path—LLM-powered semantic interpretation for ambiguous inputs or efficient keyword retrieval for precise name and entity lookups.

Together, these components provide a consistent, centrally governed semantic interface for People and Job Search, boosting relevance, simplifying the system, and enabling Semantic Search to scale across LinkedIn's global traffic.

Small language model ranking

The ranking module of semantic search utilizes a Small Language Model (SLM) to estimate how relevant a user's search query q is to each retrieved job_i . The SLM follows a decoder-only architecture. For example, for job search we represent the structured attributes of a job — including its title, company, location, employment type, and remote-work status. Meanwhile people search uses information from the member's profile including their name, company information, position information, educational information and location. For each query–job pair (q, i) , we build a structured prompt defined as

Here, the *system prefix* and *suffix* contain chat-template tags and explicit instructions guiding the model to determine whether the given job matches the query. When this prompt is passed through the decoder, it produces logits corresponding to the next token. Let $logit_{yes}$ and $logit_{no}$ represent the logits for the tokens “yes” and “no,” respectively. Following prior studies, we compute:

which yields probabilities used to rank job items by their relevance to the user's query.

SLM training pipeline for relevance quality

Training of the SLM follows a multi-stage process. First, we distill the 7B-parameter teacher model into a compact 0.6B model that can generate graded relevance labels along with rationales. Next, the teacher's ordinal grades are converted into “soft labels” (p_{yes}, p_{no}) , representing probabilistic supervision.

We then perform supervised fine-tuning (SFT) to minimize the Kullback–Leibler (KL) divergence between the teacher’s soft targets and the SLM’s predicted probabilities—effectively converting the reasoning-oriented model into a binary relevance classifier:

We construct training labels by sampling real query–item pairs from user interaction logs and annotating them using a full-scale Large Language Model (LLM). Each pair is evaluated through a structured prompt of the following format:

```
[CRITERIA]: <matching guidelines>
[EXAMPLES]: <reasoning and output format>
[QUERY]: <query text>
[CANDIDATE]: <item text>
Analyze the query–candidate pair and assess how well they align. Provi
```

The LLM’s response includes both a graded relevance score and an accompanying rationale, ensuring that the generated labels are interpretable and consistent with the defined matching criteria.

The model is trained on logged pairs query–job pairs for up to five epochs using the prompt structure. Since job descriptions dominate the input and vary substantially in length (median $\approx$ 900 tokens; maximum $>$ 2300), we truncate them to ensure the total prompt length does not exceed 2048 tokens during both training and inference. For evaluation, we use a holdout dataset labeled by the teacher model. The SLM ranks job candidates based on $p_{yes} \cdot p_{no}$, and system performance is measured using Normalized Discounted Cumulative Gain at rank 10 (NDCG@10).

Training for multiple objectives

We extend the training paradigm to predict both relevance and engagement within the model, we train a smaller cross-encoder language model (the SLM) using multi-teacher, multi-task distillation. The teachers include:

- The product policy LLM for relevance scoring.
- Other large models that predict member actions—such as job views, applies, recruiter accepts, or (in people search) view profile, connecting, messaging, or following.

For training at scale, we use multiple 1.7B teacher models. To make this feasible, we first distilled our 7B product policy model into a smaller 1.7B version, which serves as a strong but efficient teacher alongside others. During the training process, these teachers run in real time on

face liquidated damages. On top of this, I somehow still found time to invest in Rust tooling to improve security and reliability, I gave three conference talks,^[1] and I contributed to an industry working group.^[2]

Staggered by the amount of work I did in 2025, I asked myself: How much of this “productivity” was from AI?

Some of it was from working more hours than I’ve ever worked, unsustainably so.^[3] Some of it came from industry experience that allowed me to pick solutions without having to spend a lot of time doing research or experimentation. A huge part of it was from being surrounded by amazing people who led enormous parts of these projects.

In 2025, did AI generate a lot of my code, or written communication, like design documents? No. AI made prototyping easier, it accelerated routine tasks, and it helped me evaluate engineering trade-offs more efficiently, but I still authored the majority of my work.

In 2025, what AI did more than anything was act as an exceptional pairing partner who was available at any hour of the day, had access to a huge corpus of knowledge, and never got tired. AI kept me going when I otherwise would have been stuck, or when I was exhausted, one more prompt would lead to an exhilarating breakthrough, and I’d keep going.

With the release of the latest AI models near the end of 2025, software engineering as we knew it changed forever. In 2026, I expect AI will produce almost all of the code I write. I also expect to be regularly reevaluating how I approach my work to keep up with what is now possible.

Over the course of a few essays, starting with this one, I will detail how I’m experiencing and thinking about this historic shift.

Most People Have Yet to Adapt

The majority of people in the software industry have yet to realize the full potential of AI models and tools. Many people continue to work in the ways they are accustomed to. They use AI for some tasks and they are aware it is going to change their job, dramatically, they just aren’t sure how fast, or how much they will need to adapt.

Even for people who are eager to adopt AI, it is hard to get your head around exponential growth. To understand the full impact, you always need to be trying the latest models, and on broader and more difficult problems, including the full life-cycle of software engineering—product management, testing, deployment, operations, maintenance, compliance—not just

Looking ahead

We built a modern semantic search system by first establishing an LLM-based quality evaluation framework grounded in product policy. On top of this foundation, we introduced LLM-powered query understanding, semantic retrieval, and ranking models—driving double-digit improvements in search quality and member engagement. Just as importantly, we achieved this while keeping inference costs comparable to traditional RecSys models previously running in production. As we continue refining Semantic Search, our focus remains on empowering every member to discover the right opportunity, connection, or insight—at the right time.

Acknowledgements

To the more than 100 team members who’ve contributed to AI-powered job search and people search across infrastructure, AI modeling, user experience, and data science: thank you for your creativity, grit, and teamwork. This milestone belongs to all of us.

Adapting to AI: Reflections on Productivity

COLIN BRECK · 22 MAR 2026 · [SOURCE](#)



2025 was the most productive year of my career. I worked on the three largest projects I have ever worked on: migrating all services to a new cloud platform, developing a new edge computing platform, and building a distributed database. Completing just one of these projects would have made for a career-defining year, but circumstances dictated that all three projects be completed simultaneously. In addition, my team inherited three more critical projects, two of which had to be delivered by contractual deadlines for our largest customer to date, or we would

sampled production query–document pairs, producing soft probability scores that act as supervision targets. The student SLM is trained with KL divergence loss to align its output distribution with the ensemble of teachers. This setup lets us train on real-world data at scale: teacher models provide rich, nuanced probability signals, and the distilled student captures much of their reasoning capacity while remaining lightweight enough to serve millions of queries per second in production.

We have multiple steps of distillation to SLM described in Figure 3, and we show the metrics after distillation in Table 1.

	NDCG@10	Apply AUC	Click AUC
Relevance Teacher 1.7B	0.9484	-	-
Engagement Teacher 1.7B	-	0.8049	0.6772
SLM 0.6B (distilled)	0.9239	0.8007	0.6704

To make LinkedIn’s AI-powered People Search more transparent, we introduced semantic, context-aware snippets that show *why* a result matches a member’s query. Snippets highlight the most relevant terms and maintain low latency through lazy loading, caching. The approach uses semantic similarity between the query embedding and precomputed phrase embeddings (unigrams/bigrams) from profile text, surfacing the highest-scoring phrases as natural, human-readable snippets. In the offline pipeline, we extract phrases from each profile section (e.g., summary, experience, education), encode them into embeddings in a [Venice key-value store](#). This index is refreshed periodically to capture profile updates, with evaluation workflows ensuring quality and readability. At search time, the snippetting midtier receives the query embedding and candidate profiles, fetches stored phrase embeddings, computes cosine similarity, selects top phrases, and expands them into readable snippets using simple heuristics. The final output is a ranked list of profiles paired with snippets and highlighting metadata.

To improve transparency in search results, we introduced a lightweight reasoning module that explains how LinkedIn interprets a member’s query. When a query is submitted, the LLM-based understanding model extracts facets, classifies intent, and—using predefined guidelines—produces a concise “thinking state” describing how the query is parsed, along with a brief summary of the types of profiles retrieved. For unsupported or negatively intended queries, the module instead provides a clear explanation of why results may be limited. For example, “berkeley community development specialist msa professional services” becomes “Searching for community development specialist at MSA Professional Services affiliated with Berkeley.” To keep latency low, reasoning outputs are cached in Couchbase and reused for repeated or semantically similar queries.

SLM ranking model inference efficiency

We employed multiple techniques to improve efficiency of the inference system at LinkedIn including:

- Model pruning, where we remove Fully Connected Layers and remove whole transformer layers.

Model pruning

To boost inference throughput, we apply model compression via structured pruning—a technique that removes redundant components to reduce model size and computation with minimal quality loss. Because our models run on GPU infrastructure, we focus on structured pruning, which removes entire neurons, attention heads, or transformer layers so the resulting model can run efficiently on standard GPU kernels. This yields real throughput and latency gains, unlike unstructured pruning, which drops individual weights but often provides no meaningful speedup without specialized hardware.

We prune hidden neurons in Multi-layer Perceptron (MLP) blocks and attention heads in self-attention modules, and we remove full transformer layers to study trade-offs between size, efficiency, and performance ([paper](#)). Pruning MLP neurons shrinks intermediate activations, while pruning layers reduces network depth—both producing lighter, faster models. After pruning, we fine-tune the model to recover any accuracy loss, ensuring efficiency improvements do not compromise quality.

Context pruning

Item descriptions are long (median ~900 tokens and up to 2,300) making them over 94% of the SLM prompt and causing ~10% of inputs to be truncated at the 2,048-token limit. Removing descriptions severely degrades relevance quality, confirming they carry essential semantic information. To handle this, we use a slightly larger 1.7B LLM to summarize descriptions offline, where item-specific inference can be precomputed and refreshed via streaming updates. Because job descriptions include verbose, often irrelevant details, we train the summarizer with a semantics-preserving loss and a length-aware reward ([paper](#)). We fine-tune the model using RL to produce concise summaries, balancing (1) reduced input length and (2) preserved model quality. The reward combines:

- a semantic consistency term, measured via KL divergence between the SLM’s output distributions on summarized vs. raw text, and

- a length penalty that discourages overly long summaries.

The weighting factor w controls the trade-off between brevity and fidelity, enabling summaries that retain meaning while reducing inference cost.

Embedding compression

Since the computational cost of LLM inference increases quadratically with input length, reducing the number of tokens can greatly decrease overall inference expense. On top of model pruning and summarization, we invented a text–embedding hybrid interaction architecture that condenses each item’s text into a single-token embedding generated by an encoder LLM ([paper](#)). These embeddings are then merged with other textual signals and passed to a ranker LLM for relevance estimation. Because the item embeddings are precomputed and stored in a nearline cache, the volume of text processed during online inference is significantly reduced, resulting in notable improvements in efficiency.

We jointly train the hierarchy of two language models, where one model is producing embedding of the item description and the other model does the ranking of the candidates. On the figure below we show we use two 0.6B models trained jointly. As a result we could replace most of the job description with just a single embedding. We may keep important raw fields with a limited number of text tokens such as title of the job, company name, location or member name in the input.

Overall modeling quality and throughput

In the table below we show how quality and inference throughput changed as we improved the modeling technology. As part of the modeling improvement we introduced multitask learning with over 6 tasks of member actions to the model output predictions including relevance, and, for example, were able to improve Click AUC of the model from 0.61 ([baseline](#)) to 0.67 for Job Ranking.

Setup	NDCG@10 Throughput (ITEMS/SEC/GPU)	
SLM with raw-text	0.9432	290
Pruned SLM and SUMMARIZED TEXT	0.9218	2200
SLM with EMBEDDING COMPRESSION	0.9239	22000
Embedding based retrieval (EBR) baseline	0.838	>1.6B, exhaustive search on GPU